



The Control Plane and Its IAM Role

Creating a cluster gives you a managed Kubernetes API endpoint. The control plane needs an IAM role to act in your account, and you reach that API as your own AWS identity.

IN THIS LECTURE

01 The managed endpoint

What creating a cluster actually hands you.

02 The cluster role

Why the control plane assumes a role, and its two halves.

03 API access

Authenticating and authorizing with AWS identities.

04 Addons

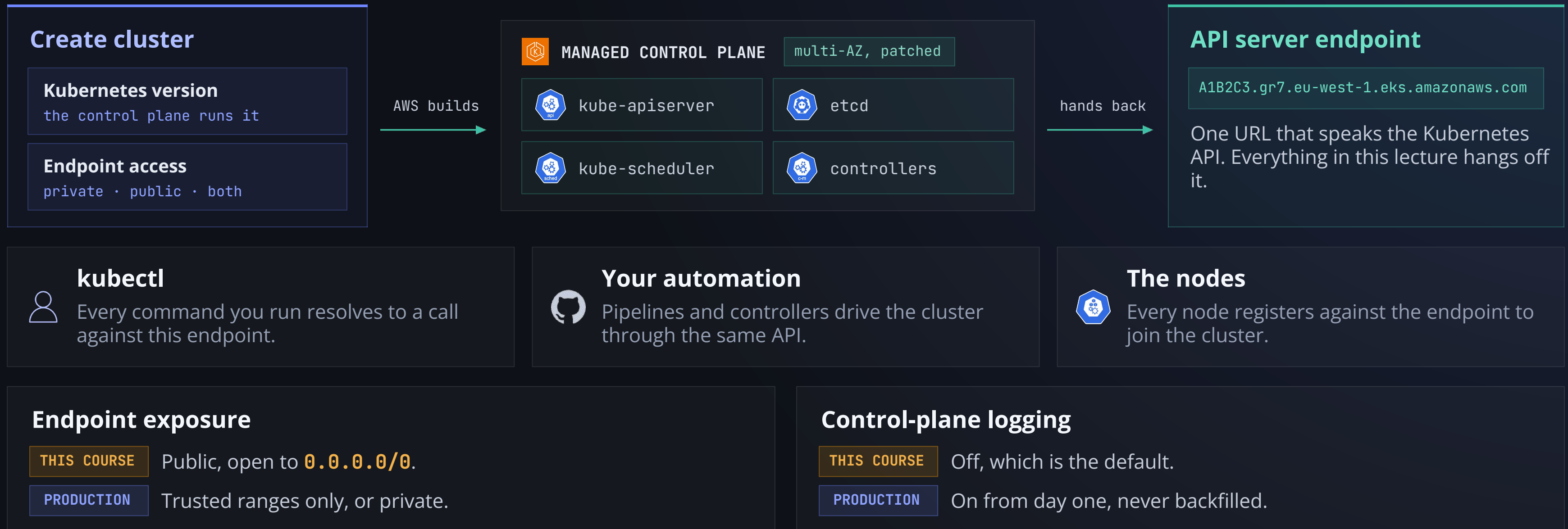
EKS-managed versus self-managed core components.



WHAT YOU GET

Creating a cluster gives you a managed API endpoint

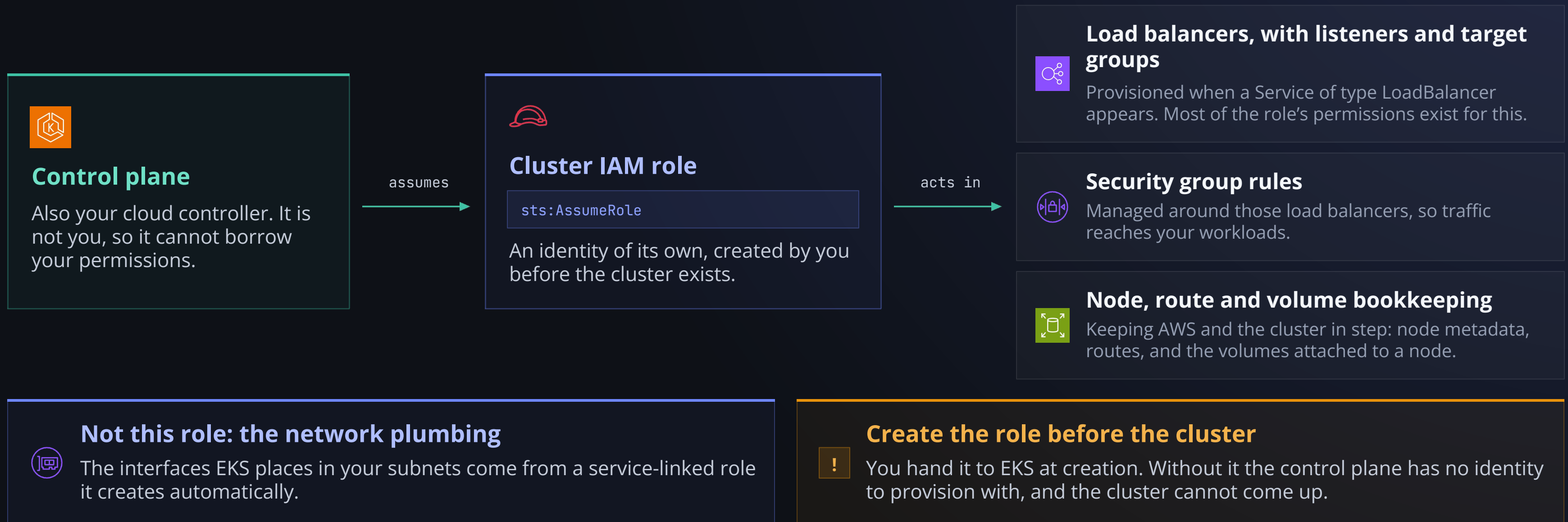
AWS runs the API server, etcd, the scheduler and the controllers across zones. You get back one endpoint, and you choose its version and how it is reached.



WHY A ROLE EXISTS

The control plane assumes a role to act in your account

Acting as a cloud controller, the control plane turns Kubernetes objects into AWS resources: a load balancer when a Service asks for one, and the rules around it. Those calls need AWS permissions.



Every role: a trust policy and a permissions policy

The same two halves build every IAM role in this course. Trust says who may assume it, permissions say what it may then do, and only the values change.



Trust policy

who may assume this role

```
"Principal": { "Service": "eks.amazonaws.com" }
"Action": "sts:AssumeRole"
```

- ✓ Names the EKS service itself, by its service principal.
- ✓ Allows exactly one action: assuming the role.
- ✓ Nobody else can assume it, and that is the point.



Permissions policy

what it may do once assumed

```
"ManagedPolicy": "AmazonEKSClusterPolicy"
"PolicyArn": "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
```

- ✓ AWS publishes a managed policy for this exact job.
- ✓ It grants the control-plane actions, and no more.
- ✓ You attach it instead of writing permissions by hand.

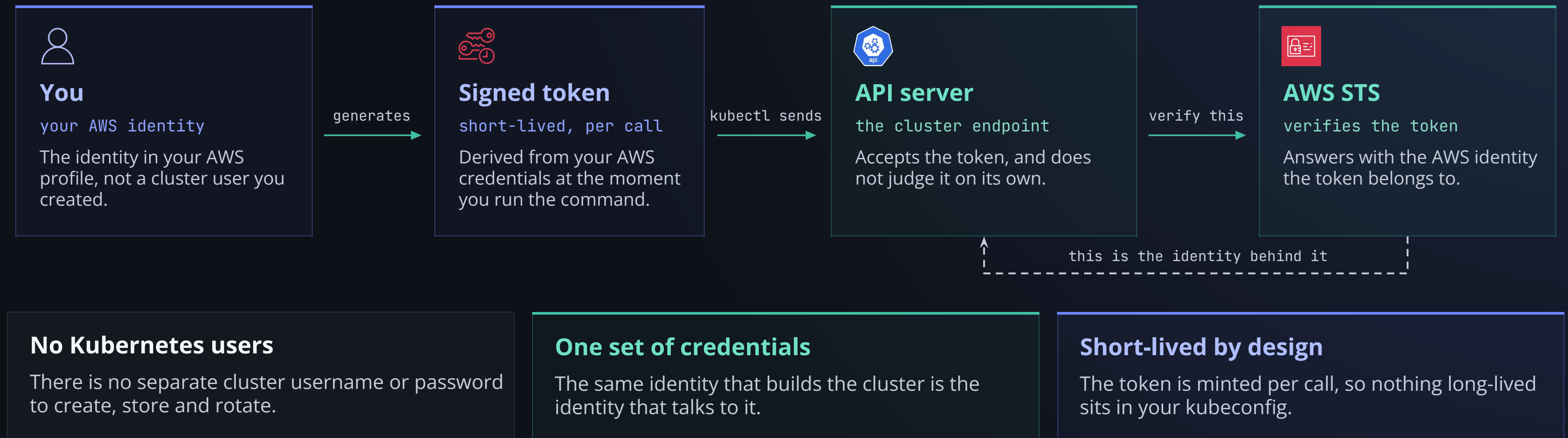
Role	Trusted principal	Permissions attached
Cluster role	eks.amazonaws.com	The AWS managed cluster policy.
Node role	ec2.amazonaws.com	Worker, networking and registry policies, built the same way.



AUTHENTICATION

You authenticate to the API as your AWS identity

EKS has no separate Kubernetes usernames. A kubectl call sends a short-lived token derived from your AWS credentials, and the API server has AWS confirm whose it is.



AUTHORIZATION

Authorization: access entries or the aws-auth ConfigMap

Authentication settles who you are. Authorization maps that AWS identity to Kubernetes permissions, and the mapping lives in one of two places.



aws-auth ConfigMap

the original mechanism

```
kubectl edit -n kube-system cm/aws-auth
```

- ✓ A Kubernetes object you edit from inside the cluster.
- ✓ Maps an AWS identity to a set of Kubernetes permissions.
- ⚠ Easy to misedit, awkward to automate, and it locks you out when it goes wrong.



EKS access entries

the newer mechanism

```
aws eks create-access-entry
```

- ✓ The same mapping, managed through the AWS API.
- ✓ Handled like any other AWS resource, so it automates cleanly.
- ✓ Auditable and repairable from outside the cluster.

CONFIGMAP

The ConfigMap alone decides.

API

Access entries alone decide.

BOTH

Either source can grant access.



The identity that creates the cluster is granted cluster administrator automatically, so your very first connection works without editing or mapping anything.

Addons: EKS-managed or self-managed

Pod networking, the service proxy and in-cluster DNS are what make a cluster usable. You choose at creation how they are run.



Pod networking

vpc-cni



Service proxy

kube-proxy



In-cluster DNS

coredns

Self-managed

the default at creation

- ✓ Bootstrapped as ordinary workloads inside the cluster.
- ✓ Full control of the manifests and the exact versions you run.
- ! You apply every upgrade yourself, and you carry the compatibility risk.

EKS-managed addons

opt in at create time

- ✓ EKS installs and version-manages each one through its own API.
- ✓ An upgrade is a version bump, applied with cluster-version awareness.
- ✓ Less maintenance, with guardrails on which versions fit your cluster.



EKS bootstraps self-managed versions unless you say otherwise, so turn that default off at create time and adopt them as managed addons. For a cluster kept simple and current, managed is the better default.

THE THREE ADDONS

Networking, service proxy, DNS

Each does one job. Miss any one and the cluster breaks in a specific, recognizable way.

Addon	Runs as	What it does	Missing it means
Pod networking vpc-cni	A pod on every node	Hands each pod a real IP address from your VPC subnets	Pods get no addresses, so nodes never report Ready
Service proxy kube-proxy	A pod on every node	Programs each node so Service traffic reaches a live pod	Traffic sent to a Service goes nowhere
DNS coredns	Ordinary pods, 2 replicas	Resolves Service and pod names to addresses in the cluster	Nothing in the cluster can resolve a name



The minimum, not the whole set

These three make a cluster usable. Production adds more, such as the EBS CSI driver so workloads can use persistent volumes.



DNS availability

EKS starts CoreDNS with two replicas and a disruption budget. Production raises the count and spreads it across nodes and zones.



Two ways to run them

A new cluster installs all three as plain manifests you update yourself. As EKS addons instead, you pick the version and AWS applies it.